

Single Logbook Entry

Logbook 1

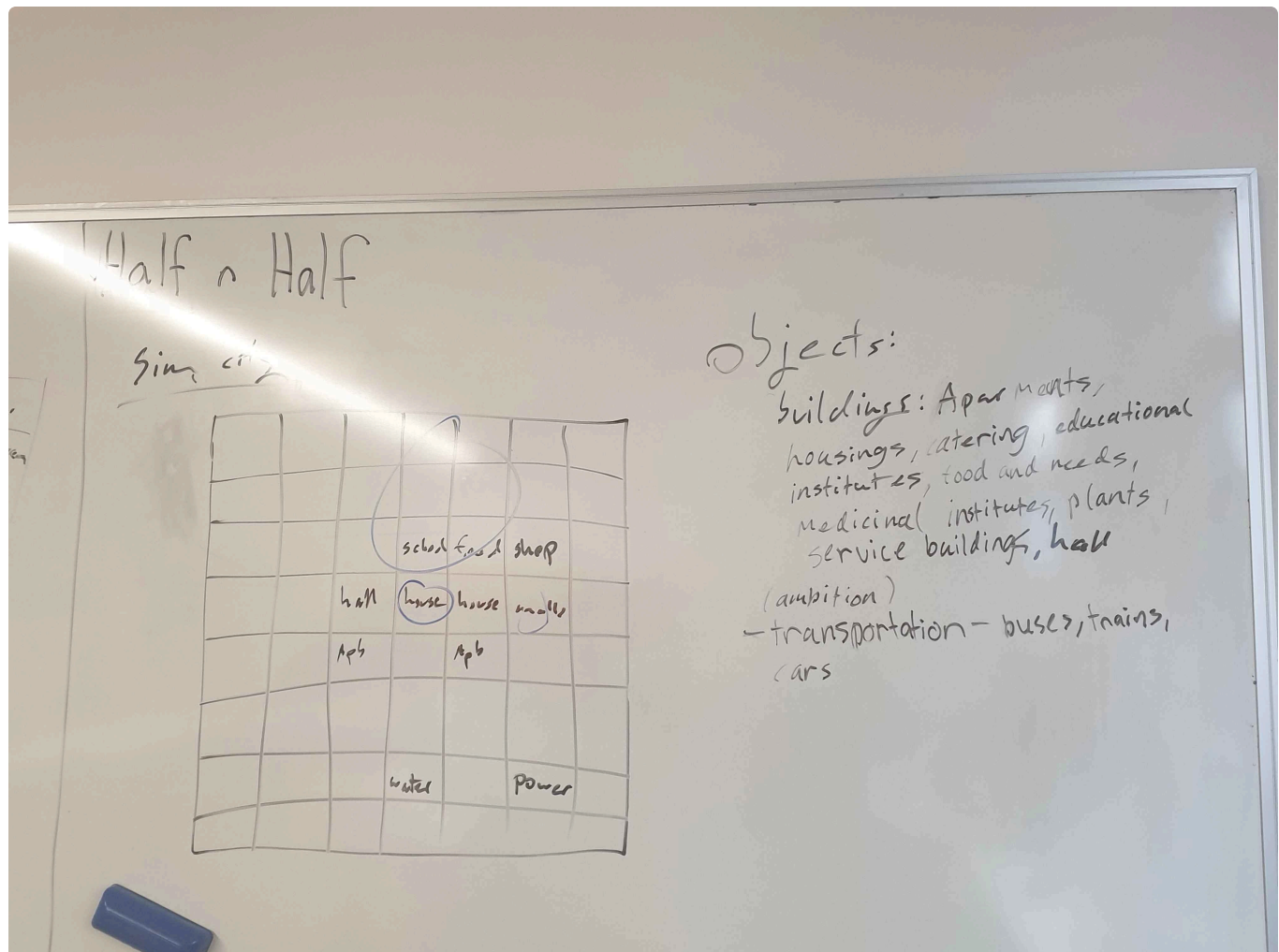
In today's practical class, we were taught how to use Git, from installation to some basic commands that enabled us to communicate with other team members. It started off with signing up for Github and creating a repository, this was shared amongst group members. From here we were able to start using some basic Git commands such as git commit, push, pull, add and log.

Today also involved a group presentation, showcasing different groups tackling today's exercise. Group 1 has been able to do a Multi-line commit text, which we weren't able to do as we were able to do single line commits. We also found out that searching messages on Git is done through the command git log --grep "KEYWORD"

We were able to achieve a full score for today's activity too, a full 7/7

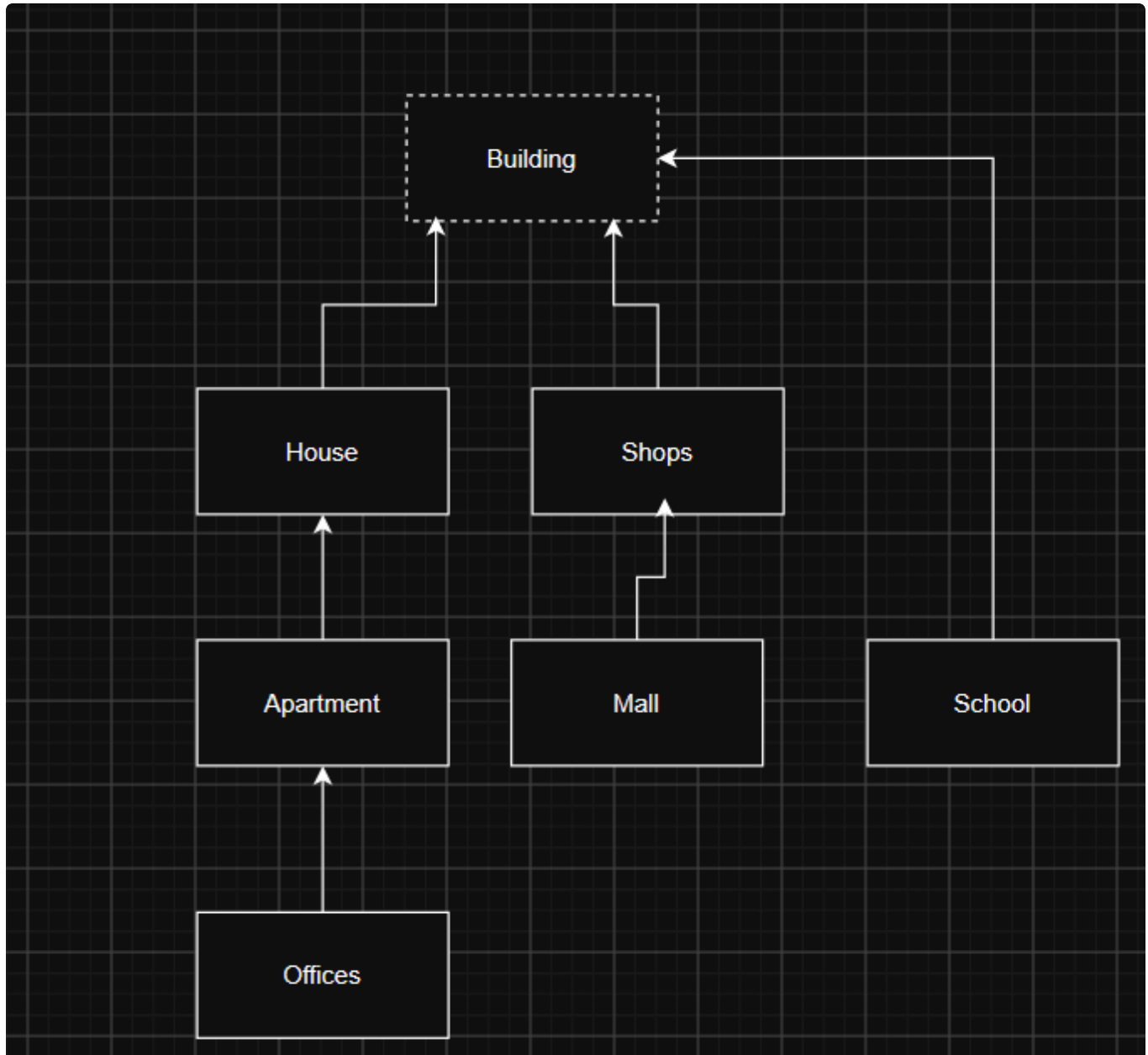
Logbook 2

Our group idea is: **Sim City**, this is what our established UML looks like:



In Sim City, we would setup a grid that would contain elements on said grid, these would represent land and different objects you would find on the land (e.g homes, towns, cities, offices, etc). Our project will simulate the natural progression of development on land. During this class, we saw numerous other groups present their ideas too, we liked 'The Williams' idea whose idea saw them simulating a battlefield, featuring terrain, troops, buffs and debuffs. It was like a game.

Logbook 3



Numerous groups went up to present their UML diagrams, I really liked team 3's diagram which was based on the development of a forest.

Though very early, we have come up with a couple of fields and methods for our superclass:

- roomCount (this will be our override as different buildings have different roomcounts)

- Population

We are currently unsure on what rules to implement into our simulation to have it naturally flow over time.

Logbook 4

Today we played around with Generics and Type Erasure.

Today's activity involved us creating intentional run time errors, we tackled the problem by creating a helper method, however, we ran into problems as we didn't know what to implement in the helper method. Other groups were able to present their solutions which we took inspiration from.

```
public class Main {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Container<String> strings = new Container<>();
        strings.add(item: "hello");
        strings.add(item: "world");
        breaks(strings);

        String first = strings.get(index: 0);
        String second = strings.get(index: 1);
        String third = strings.get(index: 2);

        System.out.println(first + " " + second);
    }

    public static void breaks(Container c){
        c.add(item: 5);
    }
}
```

Group 5 was able to write something like this which was able to create a runtime error.

Logbook 5

For the barcode exercise we were able to get to level 3. We were able to execute this via the following code blocks:

```

static void pipe () {
    System . out . print ("| ");
    caret ();
    try{

    }
    catch (Exception e){
        System . out . print ("| ");
    }

}

static void caret () {
    System . out . print ("^ ");
    //chuck_a_fit ();
    try{
        //chuck_a_fit();
    }
    catch(IllegalArgumentException e){
        System . out . print ("^ ");
    }
    //System . out . print ("^ ");

}

static void chuck_a_fit () {
    throw new myException("a exception.");

}

}
}
}

```

I liked team 4's solution which was simple and clean. For their solution, they implemented RuntimeException, they then ran a try block in pipe(), because chuck_a_fit immediately throws an error, it skips system.out.println("^"). They then printed the remaining parts of pipe in the catch statement.

Our solution saw us put the catch statement in the actual caret function. I feel like if we placed in the actual chuck a fit function it wouldnt have worked and would've just gave us an endless loop of errors.

Logbook 6

- Notes from at least two presentations by other teams.
- One technique or approach you saw that you would like to try.
- A reflection on your own submission: what went well and what you would improve.

Presentation 1 (The Williams):

- The Williams have all stats placed in one single window
- Troops move along the grid automatically
- Simulation has multiple rounds
- Round ends if their troops dies or other team's castle get destroyed
- Included are exceptions
- Different types of troops in their simulation (ranged, etc)
- They have numerous 'game states' in their simulation.
- One thing i would like to try in our project is the implementation of throws and exceptions.
- one thing i liked from this presentation is their ability to implement all the logic on one window, our one has a separate window for stats as opposed to having it all in one window.

Presentation 2 (Team B):

- Tile based grid
- Terrain on different tiles
- Natural disasters can occur during simulation
- Grid size can be manipulated over time --> this is something i like and would like to try and implement in our project.
- Two different nations that try and take over the board over time.

Presentation 3 (Starfish):

Presentation 4 (Table):

- A simulation of a planet's gravity.
- This one is my favorite one.